

IOC Container in Spring Boot

IOC (Inversion of Control) Container in **Spring Boot** is the core component that **creates, manages, and injects objects (beans)** instead of you manually creating them.

Simple Meaning

Instead of writing:

```
 new MyService()
```

Spring creates the object **for you** and gives it wherever needed.

That's why it's called **Inversion of Control** —
control of object creation moves from your code → to Spring.

What IOC Container Does

1. **Creates objects (Beans)**
 2. **Manages lifecycle** (init, destroy)
 3. **Injects dependencies** (Dependency Injection)
 4. **Handles configuration**
-

Types of IOC Containers in Spring

- **BeanFactory** → basic (rarely used directly)
- **ApplicationContext** → advanced (used in Spring Boot)

Spring Boot automatically uses **ApplicationContext**.

Quick Example

```
@Service
public class MyService {
}

@RestController
public class MyController {

    @Autowired
    private MyService service; // injected by IOC container
}
```

☞ Here Spring:

- Creates MyService object
- Injects it into MyController

✿ One-line Definition (Interview Ready)

IOC Container is the part of Spring that manages bean creation, configuration, and dependency injection.

✿ Types of IoC Containers in Spring

In **Spring Framework**, there are **two main types of IoC containers**:

1 BeanFactory (Basic Container)

- ✓ Simplest container
- ✓ Provides basic Dependency Injection
- ✓ Beans are **created lazily** (only when requested)
- ✓ Lightweight

☞ Used in: very small or memory-sensitive apps (rare in modern Spring Boot)

2 ApplicationContext (Advanced Container)

- ✓ Built on top of BeanFactory
- ✓ **Most commonly used (Spring Boot default)**
- ✓ Supports:
 - Event handling
 - Internationalization (i18n)
 - AOP integration
 - Eager bean initialization

☞ Examples:

- AnnotationConfigApplicationContext
 - ClassPathXmlApplicationContext
 - WebApplicationContext
-

🧠 Quick Interview Line

Spring has two IoC containers: BeanFactory (basic) and ApplicationContext (advanced, used in Spring Boot).

ApplicationContext in Spring

ApplicationContext is the **advanced IoC container** in the **Spring Framework** and is **the default container used by Spring Boot**.

It manages **beans, configuration, dependency injection, and application features**.

What is ApplicationContext?

 **ApplicationContext = BeanFactory + Enterprise features**

It not only creates and manages beans but also provides extra capabilities like events, i18n, and AOP.

Core Responsibilities

1 Bean Management

- Creates beans
- Injects dependencies
- Maintains bean scope (singleton, prototype)

2 Bean Lifecycle Handling

- Initialization (@PostConstruct)
- Destruction (@PreDestroy)

3 Dependency Injection

- Constructor injection
- Setter injection
- Field injection

4 Configuration Loading

- Java config (@Configuration)
 - Annotations (@Component, @Service)
 - XML (older approach)
-

Extra Features (Beyond BeanFactory)

✓ Event Publishing

- Application events & listeners

✓ Internationalization (i18n)

- MessageSource support

✓ AOP Integration

- Proxy creation for aspects

✓ Environment & Properties

- Reads application.properties / yml
-

Common Implementations

Implementation	Used For
AnnotationConfigApplicationContext	Standalone apps
ClassPathXmlApplicationContext	XML config
FileSystemXmlApplicationContext	External XML

Implementation	Used For
WebApplicationContext	Web apps
ConfigurableApplicationContext	Lifecycle control

How It Works (Flow)

- 1 Spring Boot starts
 - 2 ApplicationContext is created
 - 3 Component scan runs
 - 4 Beans are created
 - 5 Dependencies injected
 - 6 App becomes ready
-

Example

```
ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);
```

```
MyService service = context.getBean(MyService.class);
```

ApplicationContext vs BeanFactory (Quick)

Feature	BeanFactory	ApplicationContext
Bean creation	Lazy	Eager (default)
Events	✗	✓
AOP support	✗	✓
i18n	✗	✓

Feature	BeanFactory	ApplicationContext
Used in Spring Boot	✗	✓

Interview-Ready Definition

ApplicationContext is an advanced Spring IoC container that manages beans and provides enterprise features like events, AOP, and internationalization.

Spring Bean Lifecycle — Step by Step

The **Bean Lifecycle** explains **how a bean is created, initialized, used, and destroyed** inside the **Spring Framework** IoC container.

Lifecycle Phases

1 Bean Instantiation

Spring creates the object using constructor.

 `new MyBean()`

2 Dependency Injection

Spring injects required dependencies.

 via constructor / setter / field

3 Aware Interfaces (Optional)

If bean implements these, Spring provides context info:

- `BeanNameAware`
 - `BeanFactoryAware`
 - `ApplicationContextAware`
-

4 BeanPostProcessor (Before Init)

Custom logic runs before initialization.

5 Initialization Phase

Methods executed:

- @PostConstruct
- afterPropertiesSet() (InitializingBean)
- custom init-method

☞ Bean is now **ready to use**

6 BeanPostProcessor (After Init)

Spring can wrap bean with proxy (used in AOP).

7 Bean in Use

Application uses the bean normally.

8 Destruction Phase

When context closes:

- @PreDestroy
 - destroy() method
 - custom destroy method
-

Lifecycle Flow (Simple)

👉 Instantiate → Inject → Init → Use → Destroy

Example

@Component

```
public class MyBean {
```

```
    @PostConstruct
```

```
    public void init() {
```

```
        System.out.println("Bean initialized");
```

```
    }
```

```
    @PreDestroy
```

```
    public void destroy() {
```

```
        System.out.println("Bean destroyed");
```

```
    }
```

```
}
```

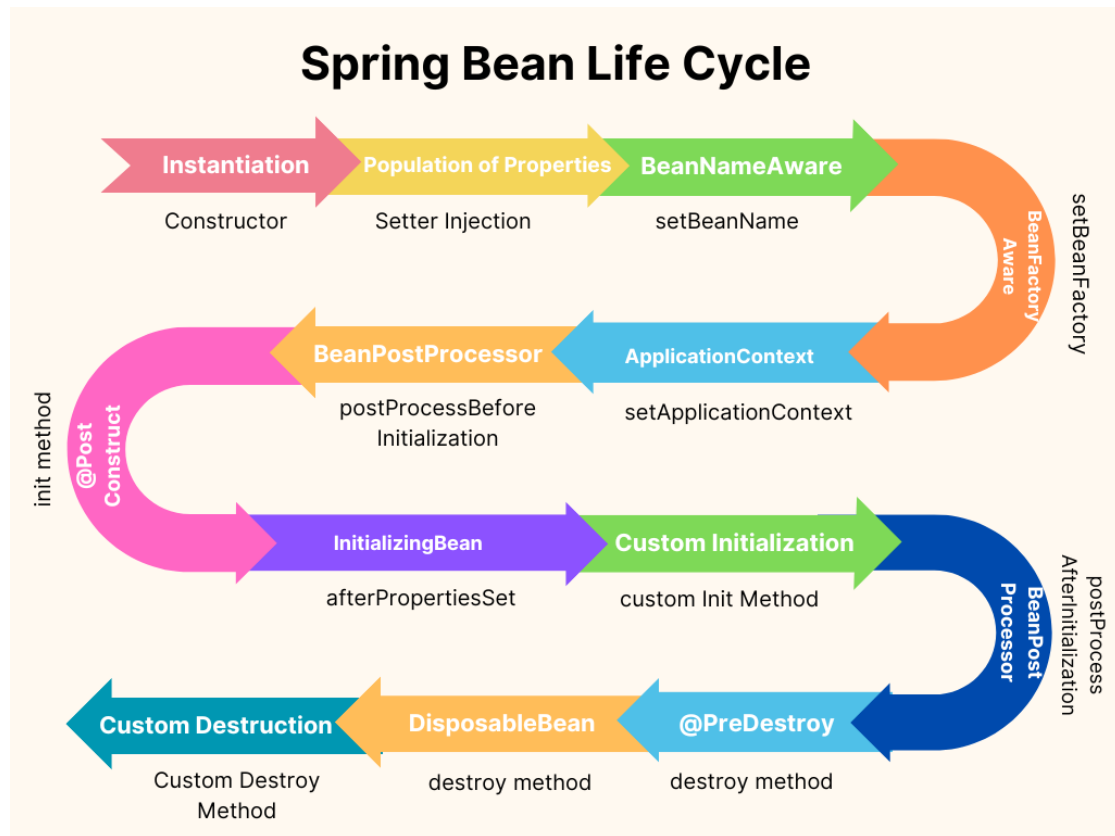
Why It Matters

- ✓ Helps in resource management
 - ✓ Used in DB connections, caches
 - ✓ Important for interviews
-

Interview One-Line

Spring Bean lifecycle includes instantiation, dependency injection, initialization, and destruction managed by the IoC container.

🔄 Complete Spring Bean Lifecycle Example



Below is a **full working example** showing almost all lifecycle steps in **Spring Boot**.

📁 1 Main Application

@SpringBootApplication

```
public class LifecycleDemoApplication {
```

```
    public static void main(String[] args) {
```

```
        ConfigurableApplicationContext context =
```

```
            SpringApplication.run(LifecycleDemoApplication.class, args);
```

```
        context.close(); // triggers destroy phase
    }
}
```

■ 2 Bean Class (Lifecycle Methods)

@Component

public class MyBean implements

```
    BeanNameAware,
    ApplicationContextAware,
    InitializingBean,
    DisposableBean {
```

```
    public MyBean() {
        System.out.println("1. Constructor called (Bean Instantiated)");
    }
```

@Autowired

```
    private MyDependency dependency;
```

@Override

```
    public void setBeanName(String name) {
        System.out.println("2. BeanNameAware: " + name);
    }
```

@Override

```
    public void setApplicationContext(ApplicationContext ctx) {
        System.out.println("3. ApplicationContextAware called");
    }
```

@PostConstruct

```
    public void postConstruct() {
        System.out.println("4. @PostConstruct called");
    }
```

```
@Override
public void afterPropertiesSet() {
    System.out.println("5. InitializingBean.afterPropertiesSet()");
}

public void customInit() {
    System.out.println("6. Custom init method");
}

@PreDestroy
public void preDestroy() {
    System.out.println("7. @PreDestroy called");
}

@Override
public void destroy() {
    System.out.println("8. DisposableBean.destroy()");
}

public void customDestroy() {
    System.out.println("9. Custom destroy method");
}
}
```

■ 3 Dependency Bean

@Component

```
public class MyDependency {
    public MyDependency() {
        System.out.println("Dependency injected");
    }
}
```

■ 4 Configuration (for custom init & destroy)

@Configuration

```
public class AppConfig {
```

```
    @Bean(initMethod = "customInit", destroyMethod = "customDestroy")
```

```
    public MyBean myBean() {
```

```
        return new MyBean();
```

```
    }
```

```
}
```

📄 Expected Console Output Order

Constructor called

Dependency injected

BeanNameAware

ApplicationContextAware

@PostConstruct

afterPropertiesSet

Custom init method

---- Application Running ----

@PreDestroy

destroy()

Custom destroy method

🌐 What This Example Covers

✓ Instantiation

✓ Dependency Injection

✓ Aware interfaces

✓ Initialization callbacks

✓ Custom init method

✓ Destruction callbacks

🎯 Interview Tip

If you explain this order clearly, it shows **strong Spring fundamentals**.

Simple Spring Project (Without Annotations — XML Configuration)

Here is a **very simple complete example** using **ApplicationContext** with **XML** (no annotations).

Project Structure

src

```
├── com.example
│   ├── App.java
│   ├── MessageService.java
│   └── MessagePrinter.java
└── applicationContext.xml
```

1 MessageService.java

```
package com.example;
```

```
public class MessageService {

    public String getMessage() {
        return "Hello from Spring XML!";
    }
}
```

2 MessagePrinter.java

```
package com.example;
```

```
public class MessagePrinter {
```



```
private MessageService messageService;

// setter injection
public void setMessageService(MessageService messageService) {
    this.messageService = messageService;
}

public void printMessage() {
    System.out.println(messageService.getMessage());
}
}
```

3 applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="messageService" class="com.example.MessageService"/>

    <bean id="messagePrinter" class="com.example.MessagePrinter">
        <property name="messageService" ref="messageService"/>
    </bean>

</beans>
```

4 Main Class (App.java)

```
package com.example;
```

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class App {

    public static void main(String[] args) {

        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        MessagePrinter printer = (MessagePrinter)
context.getBean("messagePrinter");
        printer.printMessage();
    }
}
```

► Output

Hello from Spring XML!

What This Project Shows

- ✓ IoC container creates beans
 - ✓ Dependency Injection using XML
 - ✓ ApplicationContext usage
 - ✓ No annotations at all
-

Same Project in Spring Boot using ApplicationContext + @Configuration + @Bean

This example uses **Java Configuration** (no @Service / @Component).
We explicitly define beans using **@Configuration** and **@Bean**.

Project Structure

src/main/java/com/example/demo

- |— DemoApplication.java
- |— AppConfig.java
- |— MessageService.java
- └— MessagePrinter.java

1 MessageService.java

```
package com.example.demo;

public class MessageService {

    public String getMessage() {
        return "Hello from Spring Boot Java Config!";
    }
}
```

2 MessagePrinter.java

```
package com.example.demo;

public class MessagePrinter {

    private final MessageService messageService;
```

```
public MessagePrinter(MessageService messageService) {
    this.messageService = messageService;
}

public void printMessage() {
    System.out.println(messageService.getMessage());
}
}
```

3 Configuration Class (AppConfig.java)

☞ Here we manually register beans in **ApplicationContext**

```
package com.example.demo;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

```
@Configuration
```

```
public class AppConfig {
```

```
    @Bean
```

```
    public MessageService messageService() {
```

```
        return new MessageService();
```

```
    }
```

```
    @Bean
```

```
    public MessagePrinter messagePrinter(MessageService messageService) {
```

```
        return new MessagePrinter(messageService);
```

```
    }
```

```
}
```

4 Main Class (DemoApplication.java)

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {

        ApplicationContext context =
            SpringApplication.run(DemoApplication.class, args);

        MessagePrinter printer = context.getBean(MessagePrinter.class);
        printer.printMessage();
    }
}
```

► Output

Hello from Spring Boot Java Config!

What Happens Internally

- 1 Spring Boot starts
- 2 **ApplicationContext created**
- 3 @Configuration class loaded
- 4 @Bean methods executed
- 5 Beans registered in container
- 6 Dependency injected automatically

Interview One-Line

In Spring Boot, ApplicationContext can be configured using @Configuration and @Bean to manually define and manage beans.
